PRODUCT REQUIREMENTS DOCUMENT

Bay Area Creative Dancers
Website Revamp

~ Foster the Love of Dance ~

Client	Bay Area Creative Dancers (BACDA) — 501(c) non-profit
Prepared by	YantraIQ Solutions
Framework	axiomicai/template-repo (multi-agent swarm)
Target stack	Next.js 14 · Supabase · Netlify · GitHub Actions
Document version	1.0
Status	Ready for Claude Agent execution

Table of Contents

1. Executive Summary

1.1 Project Overview

Bay Area Creative Dancers (BACDA) is a San Francisco Bay Area non-profit dance organization led by artistic director Dalia Sen. BACDA produces classical, contemporary, and fusion Indian dance productions and has been credited with major events including NABC 2009, NABC 2012, NABC 2017, Kala Mahotsav (2014/2015), Bangamela 2019, Raabdta, Bodhayon, and the 2015 opening ceremony for Indian Prime Minister Narendra Modi's Bay Area visit.

The current website (bayareacreativedancers.org) is a static HTML site hosted on GoDaddy shared hosting via cPanel. It presents dated, cluttered content with broken carousel behavior, inconsistent styling across pages, and no content management capability. The organization needs a modern, performant, and easily-maintained web presence that reflects the caliber of its productions.

1.2 Project Goals

- Deliver a visually distinctive, modern website that honors classical Indian dance aesthetics while feeling contemporary
- Move all media (videos, photo galleries) to YouTube channel embeds and Google Drive / Supabase Storage — zero large binaries in the git repository
- Provide a password-protected admin dashboard (Supabase Auth) so Dalia Sen or a delegated volunteer can edit events, testimonials, team members, and home-page content without developer involvement
- Fully automated CI/CD: every push to main triggers a Netlify production deploy via GitHub Actions
- Build the entire project using the axiomicai/template-repo multi-agent swarm harness — the output of this PRD is the input to a Claude-powered agent team

1.3 Success Criteria

Criterion	Target
Lighthouse Performance score (mobile)	≥ 90
Lighthouse Accessibility score	≥ 95
Lighthouse SEO score	100
Lighthouse Best Practices score	≥ 95
LCP (Largest Contentful Paint, p75)	≤ 2.5s
CLS (Cumulative Layout Shift)	≤ 0.1
INP (Interaction to Next Paint)	≤ 200ms

Criterion	Target
Time to First Contentful Paint	≤ 1.5s on 4G
Admin content update time	≤ 60 seconds from edit to live (ISR)
Lines of code in largest React component	≤ 300
Test coverage (admin mutations)	≥ 80%
Deployment: push to main → production live	≤ 5 minutes
Mobile responsive breakpoints	320px / 768px / 1024px / 1440px
JSON-LD schema validity	100% pass Google Rich Results Test
Organic ranking target (day 90)	Top 3 for 'Bay Area Creative Dancers', 'BACDA'

2. Current State Audit

2.1 Existing Pages

Page	URL	Issues Identified
Home	/index.html	Hard-coded event promos (2023 dates still visible in 2026); broken Facebook embed; inline styles
About	/about.html	Non-functional Bootstrap 3 carousel (prev/next links go to anchors); hardcoded BACD philosophy text; no structured team section
Events	/events.html	Mix of upcoming and past events with no clear separation; dead links; duplicated content blocks
Testimonials	/testimonials.html	Static testimonial text; no source attribution structure
Contact	/contact.html	mailto: link only; no form; no Google Maps; no social

2.2 Content to Preserve

The following content is valuable and must be carried forward into the new CMS schema:

- BACD acronym story: Bridge, Alive, Cherish, Dance (from the About page)
- Dalia Sen's biography and production credits
- Historical event archive: NABC 2009 / 2012 / 2017, Kalamahotsav 2014/2015, PM Modi Opening Ceremony 2015, Chirosokha 2015, Chitra (EnActe) 2017, Raabdta, Bodhayon, Banga Mela 2019, West Coast Theater Festival, Spring India Festival, OMG 2014, Tasher Desh, Tanushree Shankar workshop 2023, Mallika Sarabhai workshop 2023, Kingdom Of Dreams 2025
- Partner choreographer mentions: Tanushree Shankar, Mallika Sarabhai, Debojyoti Mishra, Sudarshan Chakroborty, Sanjib Bhattacharya, Subhen Khatua, Ranjita Chakraborty, Ballori Sengupta, Anuradha Mookerjee, Poushali Banerjee
- Contact information: contactus@bayareacreativedancers.org
- External ticketing links: tugoz.com/bacda2025 (Kingdom Of Dreams)

2.3 Content to Discard

- All inline HTML styling, Bootstrap 3 scaffolding, jQuery carousel code
- Stale event promos (2023 workshops treated as upcoming)
- The /img/ directory of large JPGs — these will move to YouTube/Drive/Supabase Storage
- Broken Facebook event embeds and expired RSVP links

3. Target Architecture

3.1 Tech Stack

Layer	Technology	Rationale
Frontend framework	Next.js 14 (App Router)	SSG + ISR for blazing performance; easy Netlify integration
Language	TypeScript (strict mode)	Type safety across agent-generated code
Styling	Tailwind CSS + shadcn/ui	Consistent design tokens; accessible primitives
Animation	Framer Motion	Subtle scroll/stagger animations for dance aesthetic
Database & Auth	Supabase (Postgres + Row Level Security)	Free tier sufficient; one source of truth for content and admin auth
Media storage	Supabase Storage + external embeds	Photos in Storage bucket; videos as YouTube embeds
Forms	React Hook Form + Zod	Type-safe client/server validation
Email (contact form)	Resend or Supabase Edge Function → SMTP	Transactional email from contact form
Hosting	Netlify	Zero-config Next.js support; free tier; instant rollback
DNS	GoDaddy (current) → CNAME to Netlify	No registrar migration needed
CI/CD	GitHub Actions + Netlify Deploy	Tests, linting, preview deploys on PRs
Analytics	Plausible OR Vercel Analytics	Privacy-first, cookieless, no cookie banner needed
Error monitoring	Sentry (free tier)	Production error visibility
Video embeds	lite-youtube-embed	224x smaller than raw YouTube iframe — critical for CWV
SEO helpers	next/font, next/image, app/sitemap.ts, app/robots.ts	Built into Next.js 14 — zero extra deps
Structured data	schema-dts (TypeScript types for schema.org)	Type-safe JSON-LD

3.2 System Diagram

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                              VISITOR BROWSER                                 │
│   (bayareacreativedancers.org / new.bayareacreativedancers.org)              │
└──────────────────────────────────┬───────────────────────────────────────────┘
                                   │ HTTPS
                                   ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│                        NETLIFY EDGE (CDN + SSR)                              │
│   • Next.js 14 App Router                                                    │
│   • Static pages + ISR (revalidate: 60s)                                     │
│   • Server components fetch from Supabase at build / request time            │
└──────────────────────────────────┬───────────────────────────────────────────┘
                                   │
                  ┌────────────────┼────────────────┐
                  ▼                ▼                ▼
        ┌──────────────┐   ┌──────────────┐   ┌──────────────┐
        │ SUPABASE     │   │ YOUTUBE      │   │ GOOGLE DRIVE │
        │ Postgres     │   │ (embed iframes)│ (public folder)│
        │ Auth + RLS   │   │              │   │              │
        │ Storage      │   └──────────────┘   └──────────────┘
        └──────┬───────┘
               │ authenticated admin requests
               ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│                          ADMIN DASHBOARD                                     │
│   /admin/* (protected by Supabase Auth middleware)                           │
│   • CRUD events, testimonials, team, hero content                            │
│   • Upload photos to Supabase Storage                                        │
│   • Paste YouTube URLs — auto-parse videoId                                  │
└──────────────────────────────────────────────────────────────────────────────┘

                        DEPLOY PIPELINE
┌──────────────────────┐   push   ┌──────────────────────┐  webhook  ┌──────────────┐
│  Developer /         │──────────►│  GitHub (main)       │──────────►│  Netlify     │
│  Agent Swarm         │          │  + Actions CI        │           │  Deploy      │
└──────────────────────┘          └──────────────────────┘           └──────────────┘
```

3.3 Key Architectural Decisions

ADR-001: No database for media files

Videos live on YouTube (BACDA channel); photos live in Supabase Storage or public Google Drive folders. The Postgres database stores URLs, not binaries. This keeps the database tiny, backup trivial, and bandwidth costs zero.

ADR-002: ISR over full SSG

Using Incremental Static Regeneration with a 60-second revalidation window. Content edits from the admin panel appear live within 60 seconds without a full rebuild. Critical for a non-technical content owner.

ADR-003: Single admin role (no complex permissions)

BACDA is a small volunteer organization. One admin role with full CRUD is sufficient. Row Level Security in Supabase enforces admin-only writes; public reads are allowed on content tables.

ADR-004: Staging subdomain before cutover

Deploy new site to new.bayareacreativedancers.org (CNAME to Netlify) while the old cPanel site continues serving the root domain. Only cut over root DNS after full client acceptance.

4. Information Architecture

4.1 Site Map

```
/                        Home (hero + featured event + recent videos + CTA)
├── /about                 About (mission, BACD story, Dalia Sen bio, team grid)
├── /events                Events (upcoming + archive, filterable by year)
│   └── /events/[slug]     Event detail page (gallery, video, ticket link)
├── /gallery               Photo & video gallery (YouTube channel grid + photo lightbox)
├── /testimonials          Testimonials (scrollable card grid)
├── /contact               Contact (form + email + map + social)
└── /admin                 Admin dashboard (protected)
    ├── /admin/login
    ├── /admin/events
    ├── /admin/testimonials
    ├── /admin/team
    ├── /admin/home        Edit hero text, featured event, announcements
    └── /admin/media       Upload photos, link YouTube videos
```

4.2 Page-by-Page Specification

4.2.1 Home (/)

- Hero section: full-viewport image or short looping video clip, tagline "Foster the Love of Dance", CTA button linking to featured upcoming event
- Featured upcoming event card (editable via admin → home → featured_event_id)
- BACD acronym reveal: animated 4-card grid (Bridge, Alive, Cherish, Dance) with subtle fade-in on scroll
- Recent videos: embed 3 latest videos from YouTube channel (fetched at build time via YouTube Data API v3, or manually curated in admin)
- Testimonial carousel (3 featured testimonials with swipe/pagination)
- Email newsletter signup (optional — store in Supabase newsletter_subscribers table)
- Footer: contact email, social links, copyright

4.2.2 About (/about)

- Hero image of a past performance
- Mission statement paragraph (editable via admin)
- BACD acronym full story (4 sections, one per letter)
- Dalia Sen featured bio card (photo, title, long bio, list of credits as chips)
- Team grid: other volunteers / collaborators (name, role, optional photo)
- Photo strip: 6-8 curated performance photos with lightbox

4.2.3 Events (/events)

- Tab or segmented control: "Upcoming" | "Past Performances"

- Upcoming: card grid sorted by date ascending. Each card = poster image + title + date + venue + "Get Tickets" button (external link to tugoz.com or Facebook event)
- Past: filterable by year (2012–present). Card grid with event poster, title, year. Clicking opens event detail page.

4.2.4 Event Detail (/events/[slug])

- Event hero (poster + title + date + venue)
- Long description (markdown-rendered)
- Embedded YouTube video if available
- Photo gallery (grid with lightbox)
- Collaborators section (choreographers, guest artists)
- External ticket link if upcoming; "See highlight reel" CTA if past

4.2.5 Gallery (/gallery)

- Two tabs: "Videos" and "Photos"
- Videos: grid of YouTube video cards — fetched from channel ID via YouTube Data API, or manually curated
- Photos: masonry grid with lightbox (react-photo-album or lightGallery)

4.2.6 Testimonials (/testimonials)

- Grid of testimonial cards: quote + name + title/organization + optional photo
- Admin-editable

4.2.7 Contact (/contact)

- Contact form: name, email, subject, message (React Hook Form + Zod)
- Form submission → Supabase Edge Function → email via Resend to contactus@bayareacreativedancers.org
- Rate limit: 3 submissions per IP per hour (Supabase edge function with simple in-memory counter, or Upstash Redis if formal)
- Display email, phone (if available), and embedded Google Map of typical venue or Bay Area service area
- Social links: Facebook, YouTube, Instagram

5. Database Schema (Supabase / Postgres)

5.1 Schema Overview

Nine tables. All content tables have public SELECT via RLS and admin-only INSERT/UPDATE/DELETE. Timestamps (created_at, updated_at) on every table with trigger-managed updated_at.

5.2 Table Definitions

events

```
create table events (
  id             uuid primary key default gen_random_uuid(),
  slug           text unique not null,
  title          text not null,
  subtitle       text,
  description    text, -- markdown
  event_date     date not null,
  end_date       date, -- for multi-day events
  venue_name     text,
  venue_address  text,
  venue_map_url  text,
  poster_url     text, -- Supabase Storage URL
  youtube_id     text, -- single featured video
  ticket_url     text, -- external ticketing link
  ticket_cta     text default 'Get Tickets',
  status         text not null check (status in ('upcoming', 'past', 'draft')),
  is_featured    boolean default false, -- featured on home
  year           int generated always as (extract(year from event_date)) stored,
  collaborators  text[], -- array of names
  created_at     timestamptz default now(),
  updated_at     timestamptz default now()
);

create index events_status_idx on events(status);
create index events_year_idx on events(year desc);
create index events_slug_idx on events(slug);
```

event_photos

```
create table event_photos (
  id          uuid primary key default gen_random_uuid(),
  event_id    uuid not null references events(id) on delete cascade,
  storage_path text not null, -- path in Supabase Storage bucket 'gallery'
  caption     text,
  sort_order  int default 0,
  created_at  timestamptz default now()
);
```

```
create index event_photos_event_idx on event_photos(event_id, sort_order);
```

event_videos

```
create table event_videos (
  id          uuid primary key default gen_random_uuid(),
  event_id    uuid references events(id) on delete cascade,
  youtube_id  text not null,
  title       text,
  sort_order  int default 0,
  created_at  timestamptz default now()
);
```

testimonials

```
create table testimonials (
  id           uuid primary key default gen_random_uuid(),
  quote        text not null,
  author_name  text not null,
  author_title text,
  author_photo_url text,
  is_featured  boolean default false, -- show on home
  sort_order   int default 0,
  created_at   timestamptz default now(),
  updated_at   timestamptz default now()
);
```

team_members

```
create table team_members (
  id          uuid primary key default gen_random_uuid(),
  name        text not null,
  role        text not null, -- e.g., "Artistic Director", "Choreographer"
  bio         text,
  photo_url   text,
  credits     text[], -- array of production credits
  is_lead     boolean default false, -- e.g., Dalia Sen
  sort_order  int default 0,
  created_at  timestamptz default now(),
  updated_at  timestamptz default now()
);
```

home_content (singleton)

```
create table home_content (
  id                uuid primary key default gen_random_uuid(),
  singleton_key     text unique default 'home' check (singleton_key = 'home'),
  hero_headline     text not null,
  hero_subheadline  text,
  hero_image_url    text,
```

```
  hero_video_url    text, -- optional background video
  featured_event_id uuid references events(id),
  mission_statement text,
  updated_at        timestamptz default now()
);

-- Seed single row
insert into home_content (hero_headline, hero_subheadline)
values ('Bay Area Creative Dancers', 'Foster the Love of Dance');
```

gallery_videos (standalone, not tied to event)

```
create table gallery_videos (
  id          uuid primary key default gen_random_uuid(),
  youtube_id  text not null unique,
  title       text,
  description text,
  sort_order  int default 0,
  created_at  timestamptz default now()
);
```

contact_submissions

```
create table contact_submissions (
  id          uuid primary key default gen_random_uuid(),
  name        text not null,
  email       text not null,
  subject     text,
  message     text not null,
  ip_address  inet,
  user_agent  text,
  created_at  timestamptz default now()
);

create index contact_submissions_created_idx on contact_submissions(created_at desc);
```

newsletter_subscribers

```
create table newsletter_subscribers (
  id           uuid primary key default gen_random_uuid(),
  email        text unique not null,
  subscribed_at timestamptz default now(),
  unsubscribed_at timestamptz
);
```

5.3 Row Level Security Policies

```
-- Enable RLS on all tables
alter table events enable row level security;
```

```
alter table event_photos enable row level security;
alter table event_videos enable row level security;
alter table testimonials enable row level security;
alter table team_members enable row level security;
alter table home_content enable row level security;
alter table gallery_videos enable row level security;
alter table contact_submissions enable row level security;
alter table newsletter_subscribers enable row level security;

-- Public read access on content tables (hide drafts)
create policy "public reads events" on events
  for select using (status != 'draft');
create policy "public reads event_photos" on event_photos for select using (true);
create policy "public reads event_videos" on event_videos for select using (true);
create policy "public reads testimonials" on testimonials for select using (true);
create policy "public reads team_members" on team_members for select using (true);
create policy "public reads home_content" on home_content for select using (true);
create policy "public reads gallery_videos" on gallery_videos for select using (true);

-- Admin-only writes (any authenticated user is admin in this single-role model)
create policy "admin writes events" on events
  for all using (auth.role() = 'authenticated')
  with check (auth.role() = 'authenticated');

-- Repeat the 'admin writes' pattern for every content table.

-- Contact submissions: public insert, admin read
create policy "anyone submits contact" on contact_submissions
  for insert with check (true);
create policy "admin reads contact" on contact_submissions
  for select using (auth.role() = 'authenticated');

-- Newsletter: public insert
create policy "anyone subscribes" on newsletter_subscribers
  for insert with check (true);
create policy "admin reads newsletter" on newsletter_subscribers
  for select using (auth.role() = 'authenticated');
```

5.4 Storage Buckets

Bucket	Public?	Purpose
gallery	Yes (public read)	Event photos, team photos, hero images
posters	Yes (public read)	Event poster images
private	No	Admin-only files (future use)

5.5 Admin User Setup

Admin users are created manually in Supabase dashboard (not self-serve). Initial seed:

1. Navigate to Supabase → Authentication → Users → Add user
2. Create user with email (Dalia Sen's email) and auto-generated password sent via email
3. User logs in at /admin/login and can change password via /admin/settings
4. No public signup page exists — /admin/login only

Security note

Because the RLS model treats any authenticated user as admin, DO NOT enable public signup in Supabase Auth settings. Keep email signup disabled; only manually-created users can log in.

6. Design System

6.1 Design Philosophy

BACDA's identity sits at the intersection of classical Indian arts and contemporary performance. The design should feel warm, intentional, and editorial — closer to an arts magazine than a typical non-profit site. Avoid: tropes like generic stock dance photography, cluttered layouts, and aggressive color overlays. Embrace: generous whitespace, considered typography, and photography of actual BACDA performances as the primary visual anchor.

6.2 Color Palette

Token	Hex	Usage
--ink	#1A1A1A	Primary text, headings
--cream	#FAF7F2	Page background
--white	#FFFFFF	Cards, modals
--burgundy	#8B3A3A	Primary accent (CTAs, links)
--burgundy-dark	#6B2828	Hover state
--gold	#C5A572	Secondary accent, dividers
--muted	#6B6B6B	Secondary text, captions
--border	#E8E2D5	Subtle borders
--error	#B91C1C	Form errors
--success	#15803D	Success toasts

6.3 Typography

The typography pairing intentionally avoids generic web-app defaults (Inter, Roboto, system fonts). The frontend-design skill flags these as markers of 'AI slop' aesthetics. BACDA is an arts organization — its typography must feel editorial and considered, not like a SaaS dashboard.

Role	Font	Weights	Usage
Display / Heading	Fraunces (Google Fonts)	400, 500, 600, 700	Hero, H1-H3. Soft flare serif with optical sizing
Body	Instrument Sans (Google Fonts)	400, 500, 600	Paragraph, UI, form labels — modern humanist sans with character
Accent / Quote	Fraunces italic	400, 500	Pull quotes, testimonial attributions, tagline 'Foster the Love of Dance'

Role	Font	Weights	Usage
Mono	JetBrains Mono	400	Admin code blocks, timestamps

Acceptable alternatives if Fraunces or Instrument Sans are unavailable: DM Serif Display + DM Sans (YantralQ's pairing), or Freight Display Pro + Söhne. Do NOT substitute Inter, Roboto, Poppins, or Montserrat.

Type Scale

```
--text-xs:   0.75rem  (12px)
--text-sm:   0.875rem (14px)
--text-base: 1rem     (16px)
--text-lg:   1.125rem (18px)
--text-xl:   1.25rem  (20px)
--text-2xl:  1.5rem   (24px)
--text-3xl:  2rem     (32px)
--text-4xl:  2.5rem   (40px)
--text-5xl:  3.5rem   (56px) -- hero only
--text-6xl:  4.5rem   (72px) -- desktop hero only
```

6.4 Spacing & Layout

- Container max-width: 1280px (content), 1440px (full-bleed hero)
- Gutter: 24px on mobile, 48px on desktop
- Section vertical padding: 64px mobile, 120px desktop
- Grid: 12-column with 24px gutter
- Border radius: 8px (cards), 4px (buttons), 0 (hero images — editorial square feel)

6.5 Component Patterns

- Buttons: filled (burgundy), outline (ink border), ghost (text only). Min height 44px for touch targets.
- Cards: white background, 1px border-border, 8px radius, hover lifts with shadow-md
- Form inputs: underline style (no boxed inputs), burgundy focus ring
- Navigation: sticky top bar, transparent over hero, solid on scroll

6.6 Motion Principles

- Fade + translate (8-16px) on scroll reveal using Framer Motion whileInView
- Stagger children: 80ms between items in a list
- Hover transitions: 200ms ease-out
- Respect prefers-reduced-motion: disable all non-essential animation

6.7 Accessibility Baseline

- WCAG 2.1 Level AA compliance

- Color contrast ≥ 4.5:1 for text, 3:1 for large text
- All interactive elements keyboard accessible with visible focus ring
- Alt text required on all images (admin enforces)
- Skip-to-content link at top of every page
- Semantic HTML: header, nav, main, section, article, footer
- ARIA labels on icon-only buttons

7. Admin Dashboard Specification

7.1 Access & Authentication

- Route: /admin — protected by Next.js middleware checking Supabase session
- Unauthenticated users redirected to /admin/login
- Login: email + password via Supabase Auth
- Session persistence via Supabase SSR cookies
- Logout button in admin navbar
- Password reset flow via Supabase magic link

7.2 Admin Navigation

```
┌──────────────────────────────────────────────────────────────────────┐
│ BACDA Admin          [Dalia Sen ▾]  [Logout]            │
├──────────────────────────────────────────────────────────────────────┤
│ Dashboard  Home  Events  Testimonials  Team  Media      │
│            Submissions  Newsletter  Settings             │
└──────────────────────────────────────────────────────────────────────┘
```

7.3 Screen-by-Screen

7.3.1 /admin (Dashboard)

- Welcome card with admin name
- Quick stats: # upcoming events, # unread contact submissions, # newsletter subscribers
- Recent activity feed: last 5 edits across all tables
- Quick actions: "Add Event", "Add Testimonial"

7.3.2 /admin/home

- Form to edit the single home_content row
- Fields: hero headline, subheadline, hero image upload, mission statement (markdown editor), featured event dropdown (populated from upcoming events)
- Preview button → opens / in new tab with ?preview=1

7.3.3 /admin/events

- Table view: title, date, status badge, actions (edit, delete, toggle featured)
- Filter by status (all / upcoming / past / draft)
- Sort by date
- "+ New Event" button → /admin/events/new
- Bulk actions: delete selected, change status

7.3.4 /admin/events/[id]

- Full event editor with all fields from the events table
- Slug auto-generated from title with manual override
- Markdown editor for description (use @uiw/react-md-editor or TipTap)
- Poster upload → Supabase Storage bucket 'posters'
- Photo gallery section: drag-drop upload multiple photos → bucket 'gallery', auto-creates event_photos rows
- YouTube video link input — auto-extracts videoId from any YouTube URL format
- Save / Save as Draft / Delete buttons

7.3.5 /admin/testimonials

- Card-style list with quote preview, author, featured badge
- Inline edit (modal) or full-page edit
- Drag to reorder (sort_order)

7.3.6 /admin/team

- Grid of team member cards
- Add/Edit modal: name, role, bio, photo upload, credits (tag input)
- Drag to reorder; mark as "lead" to feature at top of About page

7.3.7 /admin/media

- Gallery videos: paste YouTube URL, enter title/description, save
- Storage browser: view all files in 'gallery' and 'posters' buckets, delete unused

7.3.8 /admin/submissions

- Table of contact_submissions: name, email, subject, received_at
- Click row to view full message
- Mark as read (client-side state + optional 'read_at' column added in migration 02)

7.3.9 /admin/newsletter

- List of newsletter_subscribers with export to CSV button
- (No sending functionality in v1 — export and paste into Mailchimp/Buttondown)

7.3.10 /admin/settings

- Change password
- View Supabase project ID, YouTube channel ID (read-only, set via env)

7.4 Admin UX Principles

- Every destructive action (delete) requires confirmation dialog
- Every save shows toast: "Saved" (green) or "Error: [message]" (red)
- Auto-save drafts on event editor every 30 seconds
- Optimistic UI: updates appear instantly, rollback on error
- Never leave the user on a blank screen — show skeleton loaders during fetch

8. CI/CD & Deployment Pipeline

8.1 Pipeline Overview

```
Developer/Agent → Git Push → GitHub
                                |
                 ┌──────────────┼──────────────┐
                 ▼              ▼              ▼
       GitHub Actions      Netlify Build      (PR Only)
       — lint              — next build       Netlify Preview
       — typecheck         — deploy prod      Deploy
       — unit tests        (on main merge)
       — build check

       All checks must pass → branch protection blocks merge
```

8.2 GitHub Actions Workflow

File: .github/workflows/ci.yml

```yaml
name: CI

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
      - run: npm ci
      - name: Lint
        run: npm run lint
      - name: Typecheck
        run: npm run typecheck
      - name: Test
        run: npm run test -- --coverage
      - name: Build
        run: npm run build
        env:
          NEXT_PUBLIC_SUPABASE_URL: ${{ secrets.NEXT_PUBLIC_SUPABASE_URL }}
          NEXT_PUBLIC_SUPABASE_ANON_KEY: ${{ secrets.NEXT_PUBLIC_SUPABASE_ANON_KEY }}
      - name: Upload coverage
        uses: codecov/codecov-action@v4
```

```
        if: success()

  lighthouse:
    if: github.event_name == 'pull_request'
    needs: quality
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Wait for Netlify Preview
        uses: jakepartusch/wait-for-netlify-action@v1
        with:
          site_name: 'bacda-preview'
          max_timeout: 300
      - name: Lighthouse CI
        uses: treosh/lighthouse-ci-action@v11
        with:
          urls: ${{ steps.netlify.outputs.url }}
          uploadArtifacts: true
          budgetPath: './lighthouse-budget.json'
```

8.3 Netlify Configuration

File: netlify.toml

```
[build]
  command = "npm run build"
  publish = ".next"

[[plugins]]
  package = "@netlify/plugin-nextjs"

[build.environment]
  NODE_VERSION = "20"
  NEXT_TELEMETRY_DISABLED = "1"

[context.production.environment]
  NEXT_PUBLIC_SITE_URL = "https://bayareacreativedancers.org"

[context.deploy-preview.environment]
  NEXT_PUBLIC_SITE_URL = "$DEPLOY_PRIME_URL"

[[headers]]
  for = "/*"
  [headers.values]
    X-Frame-Options = "DENY"
    X-Content-Type-Options = "nosniff"
    Referrer-Policy = "strict-origin-when-cross-origin"
    Permissions-Policy = "camera=(), microphone=(), geolocation=()"

[[redirects]]
  from = "/index.html"
  to = "/"
  status = 301
```

```
[[redirects]]
  from = "/about.html"
  to = "/about"
  status = 301

[[redirects]]
  from = "/events.html"
  to = "/events"
  status = 301

[[redirects]]
  from = "/testimonials.html"
  to = "/testimonials"
  status = 301

[[redirects]]
  from = "/contact.html"
  to = "/contact"
  status = 301
```

8.4 Environment Variables

Variable	Scope	Purpose
NEXT_PUBLIC_SUPABASE_URL	Client + Server	Supabase project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY	Client + Server	Public anon key (RLS-protected)
SUPABASE_SERVICE_ROLE_KEY	Server only	Admin operations (edge functions)
RESEND_API_KEY	Server only	Transactional email from contact form
YOUTUBE_API_KEY	Server only (build)	Fetch latest videos from channel
YOUTUBE_CHANNEL_ID	Server only	BACDA YouTube channel ID (UCPYZ8dOpCwy-bFLRqoiX90g)
FB_OEMBED_TOKEN	Server only	Facebook/Instagram oEmbed (for IG post metadata)
INSTAGRAM_ACCESS_TOKEN	Server only (optional)	Only if upgrading to IG Basic Display API
NEXT_PUBLIC_SITE_URL	Client + Server	Canonical site URL
NEXT_PUBLIC_PLAUSIBLE_DOMAIN	Client	Plausible analytics domain (if using Plausible)
SENTRY_DSN	Client + Server (optional)	Error monitoring

8.5 Branch Protection Rules

- main branch: require pull request, require CI to pass, require 1 approval, no force push, no deletion
- develop branch (optional): require CI pass, allow force push for rebases

8.6 Deploy Environments

Environment	URL	Trigger
Production	bayareacreativedancers.org	Push to main
Staging	new.bayareacreativedancers.org	Push to develop (optional)
Preview	deploy-preview-{N}--bacda.netlify.app	Every PR

9. Multi-Agent Swarm Build Plan

9.1 Framework

This project will be built using the axiomicai/template-repo multi-agent harness. The harness provides:

- An orchestrator agent that plans waves and dispatches tasks to specialist agents
- Per-task CLAUDE.md contextual declarations that scope each agent's knowledge
- Parallel execution of independent tasks within a wave
- Wave-based sequencing — Wave N must complete before Wave N+1 begins

9.2 Agent Roster

Agent	Role	Primary outputs
orchestrator	Plans waves, assigns tasks, verifies handoffs	Wave plan, task tickets
scaffolder	Next.js + Tailwind + shadcn bootstrap	Repo skeleton, config files
db-architect	Supabase schema, RLS, migrations	SQL migration files
backend-dev	Server actions, API routes, edge functions	app/api/** and supabase/functions/**
frontend-dev	Public-facing pages and components	app/(public)/** and components/**
admin-dev	Admin dashboard pages and CRUD UI	app/(admin)/** and admin components
designer	Loads frontend-design SKILL + Section 14; owns design tokens, tailwind.config, logo system	tailwind.config.ts, ui primitives, brand/ directory
integrator	YouTube Data API, Instagram oEmbed, Facebook share, Resend, Sentry	lib/integrations/**
seo-agent	Metadata, JSON-LD schemas, sitemap, robots, keyword audit, CWV monitoring	app/sitemap.ts, app/robots.ts, lib/seo/**, per-page metadata
qa-agent	Unit tests, Playwright E2E, accessibility audits, Lighthouse CI	__tests__/** and e2e/**

Agent	Role	Primary outputs
devops-agent	GitHub Actions, Netlify config, env setup docs	.github/workflows/, netlify.toml
reviewer	Code review, PR approval, constitution check	Review reports, approved PRs
deployer	Final Netlify deploy, DNS cutover	Deploy confirmation

9.3 Wave Plan

Wave 0 — Preflight (serial, orchestrator only)

5. orchestrator reads this PRD and confirms understanding
6. orchestrator creates a TASKS.md in repo root with all tickets
7. orchestrator verifies required env vars and secrets exist

Wave 1 — Foundation (parallel)

8. scaffolder: bootstrap Next.js 14 App Router + TypeScript + Tailwind + shadcn/ui
9. db-architect: write migrations 0001_init.sql through 0009 (one per table) + seed.sql with BACDA content
10. designer: implement design tokens in tailwind.config.ts and globals.css; build base shadcn primitives (Button, Input, Card, Dialog, Toast)
11. devops-agent: create .github/workflows/ci.yml, netlify.toml, .env.example

Wave 2 — Core Build (parallel)

12. frontend-dev: implement public layout (nav, footer), /, /about, /events, /events/[slug], /gallery, /testimonials, /contact
13. admin-dev: implement /admin layout, login, middleware protection, dashboard, all CRUD screens, /admin/social, /admin/instagram-highlights
14. backend-dev: implement server actions for all admin mutations; contact form edge function with Resend
15. integrator: YouTube Data API (lib/integrations/youtube.ts), Instagram oEmbed (lib/integrations/instagram.ts), Facebook share helpers, lite-youtube-embed setup
16. seo-agent: app/sitemap.ts, app/robots.ts, JSON-LD components (Organization/Event/Person/VideoObject/BreadcrumbList), per-page generateMetadata() — reviews every frontend PR

Wave 3 — Quality Gate (parallel)

17. qa-agent: unit tests for all server actions, Zod schemas, utility functions
18. qa-agent: Playwright E2E: visitor flows, admin login + create event, contact form submission
19. qa-agent: axe-core accessibility scan on every public page, fix violations
20. reviewer: code review against the Constitution (section 9.5)

Wave 4 — Deploy & Handoff (serial)

21. deployer: first Netlify deploy to preview, smoke test, then to new.bayareacreativedancers.org
22. deployer: run final Lighthouse and axe audits, document scores
23. deployer: hand off admin credentials and content-editing guide (README_ADMIN.md) to BACDA

9.4 Root CLAUDE.md

Every agent reads the root CLAUDE.md. It declares the shared constitution:

```
# BACDA Website — Agent Constitution

## Project
Revamp of bayareacreativedancers.org as a Next.js 14 + Supabase + Netlify site.

## Non-negotiables
1. TypeScript strict mode ON. No 'any' except where unavoidable (must be commented).
2. Every database mutation goes through a server action with Zod validation.
3. Every public page must pass Lighthouse ≥ 90 (mobile) and axe with zero violations.
4. No client-side secrets. Service role key is server-only.
5. Every component ≤ 300 lines. Split if larger.
6. Tailwind classes only — no inline style={} except for dynamic values.
7. Every image has alt text. Every form has labels. Every interactive element is
   keyboard accessible.
8. Content is DB-driven. Never hardcode events or testimonials in source.

## Coding style
- File naming: kebab-case for components (event-card.tsx), camelCase for utilities
- Component pattern: server component by default, "use client" only when needed
- Error handling: wrap server actions in try/catch with structured error returns
- Imports order: React → third-party → @/lib → @/components → relative

## Git workflow
- Branch per ticket: feat/<wave>-<ticket-id>-<slug>
- Commit messages: conventional commits (feat:, fix:, chore:, docs:)
- PR title: "[Wave N] <ticket-id>: <title>"
- Every PR must pass CI and get reviewer approval before merge

## Verification before merge
- [ ] Tests pass locally
- [ ] Types check
- [ ] Lint clean
- [ ] Lighthouse (if UI change) — score recorded in PR description
- [ ] Manual smoke test on preview deploy
```

9.5 Per-Agent CLAUDE.md Example (frontend-dev)

```
# Frontend Developer — Scope
```

```
## You own
- app/(public)/**
- components/ui/** (shared primitives — coordinate with designer)
- components/sections/** (page sections)
- lib/fetchers/** (typed data fetchers from Supabase)

## You do not touch
- app/(admin)/** (admin-dev owns)
- supabase/migrations/** (db-architect owns)
- app/api/** (backend-dev owns)

## Patterns
- All pages are React Server Components unless they need interactivity
- Fetch data at the page level, pass as props to client components
- Use Suspense for streaming; show <Skeleton /> during load
- Use next/image for every image; specify width and height

## Wave 2 deliverables
- [ ] app/(public)/layout.tsx with nav + footer
- [ ] app/(public)/page.tsx (home)
- [ ] app/(public)/about/page.tsx
- [ ] app/(public)/events/page.tsx
- [ ] app/(public)/events/[slug]/page.tsx
- [ ] app/(public)/gallery/page.tsx
- [ ] app/(public)/testimonials/page.tsx
- [ ] app/(public)/contact/page.tsx
- [ ] components/sections/hero.tsx
- [ ] components/sections/event-grid.tsx
- [ ] components/sections/testimonial-carousel.tsx
- [ ] components/sections/youtube-embed.tsx
```

9.6 Handoff Protocol

- Each agent ends its task by writing a handoff note to .agent-handoffs/<ticket-id>.md
- Handoff includes: files changed, decisions made, open questions, test status
- orchestrator reads handoffs before dispatching the next wave
- reviewer reads handoffs during code review

9.7 Failure Handling

- If an agent's task fails (tests red, build broken), it writes a failure note and halts
- orchestrator decides: retry, reassign, or escalate to human
- No agent silently moves on from a failure

10. Testing Strategy

10.1 Test Pyramid

Layer	Tool	Coverage Target
Unit (utils, Zod schemas, fetchers)	Vitest	≥ 90%
Component (UI primitives)	Vitest + Testing Library	≥ 80%
Integration (server actions)	Vitest + Supabase local	100% of mutations
E2E (critical user flows)	Playwright	5 key flows
Accessibility	axe-core + Playwright	Zero violations on public pages
Performance	Lighthouse CI	Mobile ≥ 90

10.2 Critical E2E Flows

24. Visitor lands on home → sees featured event → clicks CTA → external ticket page opens
25. Visitor browses events → filters by 2019 → opens Banga Mela detail → video plays
26. Visitor submits contact form → sees success toast → admin sees submission
27. Admin logs in → creates new event → saves as draft → publishes → appears on /events within 60s
28. Admin uploads 5 photos to an event → reorders → saves → gallery reflects order

10.3 Non-Functional Testing

- Load test: 100 concurrent visitors via k6 (optional)
- Cross-browser: Chrome, Safari, Firefox, Edge (Playwright matrix)
- Mobile devices: iPhone SE, iPhone 14, Pixel 7 (BrowserStack or physical)
- Screen reader: NVDA + VoiceOver spot checks on home and contact

11. Domain, DNS & Migration Plan

11.1 Current State

- Registrar: GoDaddy
- Hosting: GoDaddy cPanel shared hosting
- DNS: Managed in GoDaddy
- Email: Check whether email is routed through GoDaddy or separate — must NOT break during cutover

11.2 Target State

- Registrar: GoDaddy (unchanged)
- Hosting: Netlify
- DNS: GoDaddy with records pointing to Netlify
- Email: Unchanged (MX records preserved)

11.3 Migration Phases

Phase 1 — Staging Setup (Week 1)

29. Create Netlify site connected to GitHub repo
30. In GoDaddy DNS, add CNAME: new → bacda-site.netlify.app
31. In Netlify, add custom domain: new.bayareacreativedancers.org
32. Wait for SSL certificate provisioning (~5 min)
33. Verify new.bayareacreativedancers.org loads the new site

Phase 2 — Content Population (Week 2)

34. Admin logs into Supabase dashboard, creates own account
35. Admin populates all events (upcoming + archive) via /admin/events
36. Admin uploads hero image, team photos, event posters
37. Admin writes testimonials and team member bios
38. Stakeholder reviews staging site, signs off

Phase 3 — Production Cutover (Week 3)

39. BACKUP: download full cPanel site to local storage (already done)
40. BACKUP: export Supabase data to SQL dump
41. Verify MX records in GoDaddy — write them down
42. Change GoDaddy DNS:
    - A record @ → Netlify load balancer (75.2.60.5)
    - AAAA record @ → Netlify IPv6 (2600:1f14:xxxx)
    - CNAME www → bacda-site.netlify.app

43. In Netlify, add custom domains: bayareacreativedancers.org and www.bayareacreativedancers.org
44. Confirm MX records are still pointing to email provider (DID NOT CHANGE)
45. Wait for SSL cert on root domain
46. Verify site loads on both apex and www
47. Verify email still works (send test email to contactus@)
48. Remove staging CNAME (new.*) after 7 days of stability

Phase 4 — Decommission (Month 2)

49. Keep cPanel hosting active for 30 days as backup
50. After 30 days stable, cancel cPanel hosting plan (if email is NOT routed through it)
51. If email IS routed through cPanel: migrate to Google Workspace or Zoho before cancelling

11.4 Rollback Plan

If production cutover fails

In GoDaddy DNS, revert A/AAAA/CNAME records to the original cPanel IP. Propagation takes 5-60 minutes depending on TTL. Keep TTL at 300 seconds during cutover week to minimize rollback time. Old cPanel site remains untouched throughout — rollback is purely a DNS change.

12. Deliverables & Handoff

12.1 Code Deliverables

- GitHub repository with full source, published to github.com/<org>/bacda-site
- Supabase project (URL and admin credentials shared via 1Password or similar)
- Netlify site (transferred to BACDA-owned account or kept under YantraIQ with billing passthrough)

12.2 Documentation

- README.md — developer quick start (clone, install, .env setup, run dev)
- README_ADMIN.md — non-technical admin guide with screenshots: how to log in, how to add an event, how to upload photos, how to change the home banner
- ARCHITECTURE.md — system overview, inspired by this PRD but updated to reflect final build
- RUNBOOK.md — common operational tasks: rotate API keys, invite new admin, restore from backup

12.3 Training

- One 60-minute screen-share walkthrough with Dalia Sen (or delegated admin)
- Recorded video of the walkthrough, stored in Google Drive
- Two weeks of async email support post-launch for admin questions

12.4 Acceptance Criteria

52. All pages from the information architecture are implemented and content-complete
53. Admin can successfully create/edit/delete an event, testimonial, team member, and home banner
54. Admin can add Instagram highlights via URL paste with thumbnail auto-fetch
55. Contact form submission arrives at contactus@bayareacreativedancers.org
56. Lighthouse mobile Performance ≥ 90, SEO = 100, Accessibility ≥ 95 on home, events, and about pages
57. Core Web Vitals: LCP ≤ 2.5s, CLS ≤ 0.1, INP ≤ 200ms measured on real mobile device
58. axe-core reports zero violations on all public pages
59. Push to main triggers Netlify production deploy; admin edits appear within 60 seconds on production
60. Site loads on bayareacreativedancers.org and www.bayareacreativedancers.org with valid SSL
61. Email to contactus@ still works post-cutover (MX records preserved)
62. sitemap.xml is accessible at /sitemap.xml and includes all public routes + dynamic events
63. robots.txt is accessible at /robots.txt and correctly blocks /admin and /api

64. All JSON-LD passes Google Rich Results Test with zero errors
65. OpenGraph preview on Facebook/WhatsApp/Slack renders the og-image and correct title
66. YouTube latest-videos feed loads on /gallery with lite-youtube-embed (< 50KB initial payload per video card)
67. Social icons in nav and footer link to confirmed handles (FB, YouTube, and IG if provided)
68. Google Search Console verified, sitemap submitted, first crawl successful
69. Plausible / Vercel Analytics is tracking page views and outbound link clicks

13. Timeline & Open Questions

13.1 Estimated Timeline

Week	Phase	Agent Waves
Week 1	Setup + staging domain live	Wave 0-1
Week 2	Public pages + admin CRUD complete	Wave 2
Week 3	Content population + QA + client review	Wave 3 + content work
Week 4	Production cutover + training + handoff	Wave 4

13.2 Open Questions for BACDA

70. CONFIRMED: YouTube channel ID UCPYZ8dOpCwy-bFLRqoiX90g (Bay Area Creative Dance Academy). Is this the correct channel, or does BACDA use a different one?
71. CONFIRMED: Facebook page facebook.com/BayAreaCreativeDanceAcademy. Is this still active and the primary FB presence?
72. Instagram handle — does BACDA have an active IG account? If yes, what is the @handle? If no, IG icon will be omitted from the site
73. Is there a TikTok, LinkedIn, or other social presence to include?
74. Does BACDA have a Google Business Profile? If not, recommend creating one for local SEO
75. Is there a Google Drive folder with high-res performance photos to seed the gallery?
76. Is contactus@bayareacreativedancers.org hosted via GoDaddy email, Google Workspace, or elsewhere? (Determines email migration risk)
77. Are there any partner organizations whose logos should appear in a 'Sponsors / Partners' footer section?
78. Should the newsletter subscription feature be included in v1, or deferred?
79. Should there be a 'Donate' CTA, and if so, connected to which payment processor (Zeffy, Donorbox, Stripe)?
80. What languages should the site support? (v1 assumption: English only. Bengali / Hindi later if requested.)
81. Who owns the BACDA brand assets (logo source files, approved photography)?
82. Does BACDA want to claim a Facebook Page access token to enable upcoming-events auto-sync from Facebook Events? Requires Page admin to generate the token.

13.3 Out of Scope for v1

- Multi-language support (i18n)
- Online class enrollment / ticketing (external providers handle this)
- Blog / news section (add in v2 if demand emerges)
- Automated newsletter sending (export CSV to external tool)

- Donations (add if requested)
- Member-only content / paywall

14. Visual Identity & Design Transformation

14.1 Mandatory: Load the frontend-design Skill First

Directive to the designer agent

Before writing a single line of tailwind.config.ts, globals.css, or any component, the designer agent MUST call the 'view' tool on /mnt/skills/public/frontend-design/SKILL.md and internalize its guidance. This skill is the authoritative design constitution for this project. Every other agent that touches UI (frontend-dev, admin-dev) must also load this skill before producing visual output.

The frontend-design skill exists specifically to prevent AI-generated interfaces from defaulting to generic, forgettable aesthetics. Its core mandates that apply to this project:

- NEVER use Inter, Roboto, Arial, or system fonts — use the distinctive pairing specified in Section 6.3
- Avoid purple gradients on white, pastel blur blobs, and other 'AI slop' visual tropes
- Commit to ONE bold aesthetic direction with precision — for BACDA, that direction is editorial/arts-magazine
- Typography, color, and spatial composition must feel intentional, not generated
- Add atmosphere: grain textures, generous whitespace, asymmetric layouts, considered motion

14.2 Aesthetic Direction for BACDA

The chosen direction is 'editorial performing-arts publication' — think The New York Times arts section, Apollo magazine, or a contemporary ballet company's playbill. Not a generic non-profit template. Not a SaaS dashboard. Not a wedding photographer's site.

Element	DO	DON'T
Hero	Full-bleed performance photography with soft grain overlay	Stock dance silhouettes or abstract shapes
Typography	Fraunces (serif) + Instrument Sans — editorial pairing	Inter + generic sans, Poppins, Montserrat
Layout	Asymmetric grids, generous margins, pull quotes	Symmetric card grids everywhere
Color	Warm cream background, ink text, burgundy accent sparingly used	Purple gradients, rainbow palettes, neon
Motion	Subtle staggered reveals, respect prefers-reduced-motion	Parallax everything, constant hover animations

Element	DO	DON'T
Imagery	Real BACDA performance photos at full resolution	Stock photography, AI-generated illustrations

14.3 BACDA Logo System

14.3.1 Source Asset

The authoritative BACDA logo is located at this path in the downloaded cPanel archive:

```
public_html/img/bacda-2020-logo.png
```

Action for the scaffolder agent:

83. Copy bacda-2020-logo.png from the cPanel download into the new repo at public/brand/bacda-logo-original.png
84. Do NOT modify, crop, or re-compress the original. Keep it as the canonical reference.
85. All derived assets (favicons, social cards, monochrome variants) must be generated from this source

14.3.2 Required Derived Assets

The designer agent must generate the following files into public/brand/ before Wave 2 UI work begins:

File	Size / Format	Purpose
bacda-logo.svg	Vector, optimized	Primary logo for nav, footer, emails
bacda-logo-mono-light.svg	Vector, single color #FAF7F2 (cream)	Monochrome variant for dark hero overlays
bacda-logo-mono-dark.svg	Vector, single color #1A1A1A (ink)	Monochrome for light backgrounds when full-color is too busy
favicon.ico	16×16, 32×32, 48×48 multi-res	Browser tab icon
favicon.svg	Vector	Modern browsers, adapts to theme
apple-touch-icon.png	180×180 PNG	iOS home screen
icon-192.png	192×192 PNG	PWA manifest
icon-512.png	512×512 PNG	PWA manifest / splash
og-image.png	1200×630 PNG	OpenGraph social share card
og-image-square.png	1200×1200 PNG	Instagram / WhatsApp preview
Vectorization note		

File	Size / Format	Purpose
The original bacda-2020-logo.pn g is a raster image. The designer agent should either (a) manually recreate it as clean SVG paths, or (b) use a tracing tool like potrace / vectorizer.ai and then clean up the output. A crisp SVG is non-negotiable — rasterized logos at small sizes look amateur.		

14.3.3 Logo Placement Rules

Location	Variant	Size	Behavior
Nav (over hero)	mono-light SVG	height 36px mobile, 44px desktop	Transparent nav, logo in cream
Nav (scrolled)	full-color SVG	height 36/44px	Solid white nav bar appears on scroll
Footer	full-color SVG	height 56px	Centered above contact info
Admin login	full-color SVG	height 64px	Above login form, centered
404 / error pages	mono-dark SVG	height 48px	Quiet presence above error message
Email (contact form reply)	PNG @ 2x	height 48px	Embedded in HTML email template

14.3.4 Clear Space & Minimum Size

- Clear space: minimum padding around the logo equals 50% of the logo's height on all sides
- Never place the logo directly against a high-contrast photo — use a scrim, solid bar, or apply the monochrome variant
- Minimum size: 24px height (favicon excepted). Below this, legibility breaks down
- Never stretch, skew, rotate, recolor (outside the two approved monochrome variants), or apply drop shadows

14.3.5 Color Extraction

The designer agent should inspect bacda-2020-logo.png and extract its dominant accent color, then verify it harmonizes with the Section 6.2 palette. If the logo's native color differs significantly from the burgundy (#8B3A3A) specified in the design system:

86. Update the --burgundy token in tailwind.config.ts to match the logo color exactly
87. Update Section 6.2 in a follow-up documentation PR
88. Verify contrast ratios still meet WCAG AA on the new color

14.4 Designer Agent CLAUDE.md

File: components/CLAUDE.md (scoped to designer's work area)

```
# Designer Agent — Scope

## Required reading BEFORE any work
1. /mnt/skills/public/frontend-design/SKILL.md  — design constitution
2. PRD Section 6 (Design System) and Section 14 (Visual Identity)
3. public/brand/bacda-logo-original.png  — the source logo
4. Old site screenshots in docs/legacy-screenshots/ — for reference of
   what NOT to reproduce

## You own
- tailwind.config.ts — design tokens
- app/globals.css — font imports, CSS variables, base styles
- public/brand/** — logo system and derived assets
- components/ui/** — shadcn primitives (Button, Input, Card, etc.)
- components/brand/logo.tsx — responsive Logo component with variants

## You do not touch
- app/**/page.tsx (frontend-dev and admin-dev own pages)
- supabase/** (db-architect owns)
- .github/** (devops owns)

## Non-negotiables
1. NEVER use Inter, Roboto, or Arial. Typography is Fraunces + Instrument Sans.
2. Load fonts via next/font/google for zero CLS.
3. Every color must be a CSS variable. No hardcoded hex in components.
4. Logo component must accept variant="color" | "mono-light" | "mono-dark"
   and size="sm" | "md" | "lg" props.
5. Dark-mode support is out of scope for v1 — build light-only.
6. Every interactive primitive has: hover, focus-visible, active, disabled states.
7. Focus ring is 2px burgundy, offset 2px. Visible on keyboard nav only.

## Deliverables (Wave 1)
- [ ] tailwind.config.ts with all Section 6 tokens
- [ ] app/globals.css with font-face and CSS variables
- [ ] public/brand/bacda-logo.svg (vectorized from PNG)
- [ ] public/brand/bacda-logo-mono-light.svg
- [ ] public/brand/bacda-logo-mono-dark.svg
- [ ] public/favicon.ico, public/favicon.svg, public/apple-touch-icon.png
- [ ] public/og-image.png
- [ ] components/brand/logo.tsx with variant + size props
- [ ] components/ui/button.tsx (filled, outline, ghost variants)
- [ ] components/ui/input.tsx (underline style per Section 6.5)
```

```
- [ ] components/ui/card.tsx
- [ ] components/ui/dialog.tsx
- [ ] components/ui/toast.tsx
- [ ] Storybook or preview page at app/(dev)/ui/page.tsx showing all primitives

## Verification
- [ ] Screenshot every primitive at default / hover / focus / disabled states
- [ ] Run Lighthouse on the preview page — score ≥ 95 across all categories
- [ ] Verify logo renders crisply at 24px, 48px, 96px, 192px sizes
- [ ] Verify color contrast: body text ≥ 4.5:1, muted text ≥ 4.5:1 on cream
```

14.5 Logo React Component Contract

Every page that renders the logo uses this single component — no raw <img> tags for the logo elsewhere. File: components/brand/logo.tsx

```
import Image from 'next/image';
import { cn } from '@/lib/utils';

type LogoVariant = 'color' | 'mono-light' | 'mono-dark';
type LogoSize = 'sm' | 'md' | 'lg' | 'xl';

const SIZES: Record<LogoSize, { width: number; height: number }> = {
  sm: { width: 96,  height: 32 },
  md: { width: 132, height: 44 },
  lg: { width: 168, height: 56 },
  xl: { width: 192, height: 64 },
};

const SRC: Record<LogoVariant, string> = {
  'color':      '/brand/bacda-logo.svg',
  'mono-light': '/brand/bacda-logo-mono-light.svg',
  'mono-dark':  '/brand/bacda-logo-mono-dark.svg',
};

export function Logo({
  variant = 'color',
  size = 'md',
  className,
  priority = false,
}: {
  variant?: LogoVariant;
  size?: LogoSize;
  className?: string;
  priority?: boolean;
}) {
  const { width, height } = SIZES[size];
  return (
    <Image
      src={SRC[variant]}
      alt="Bay Area Creative Dancers"
      width={width}
      height={height}
```

```
      priority={priority}
      className={cn('select-none', className)}
    />
  );
}
```

14.6 Favicon & Metadata Configuration

File: app/layout.tsx

```
export const metadata: Metadata = {
  metadataBase: new URL('https://bayareacreativedancers.org'),
  title: {
    default: 'Bay Area Creative Dancers',
    template: '%s — Bay Area Creative Dancers',
  },
  description: 'A San Francisco Bay Area non-profit dance organization. Foster the Love of
Dance.',
  icons: {
    icon: [
      { url: '/favicon.ico', sizes: 'any' },
      { url: '/favicon.svg', type: 'image/svg+xml' },
    ],
    apple: '/apple-touch-icon.png',
  },
  manifest: '/manifest.webmanifest',
  openGraph: {
    title: 'Bay Area Creative Dancers',
    description: 'Foster the Love of Dance',
    url: 'https://bayareacreativedancers.org',
    siteName: 'Bay Area Creative Dancers',
    images: [{ url: '/og-image.png', width: 1200, height: 630 }],
    type: 'website',
  },
  twitter: {
    card: 'summary_large_image',
    title: 'Bay Area Creative Dancers',
    description: 'Foster the Love of Dance',
    images: ['/og-image.png'],
  },
};
```

14.7 Agent Instructions Summary

For the orchestrator

When dispatching Wave 1, inject this note into the designer agent's task prompt: 'Before producing any code, you must (1) call the view tool on /mnt/skills/public/frontend-design/SKILL.md, (2) read PRD Section 14 in full, (3) inspect public/brand/bacda-logo-original.png. Only after these three reads may you begin

generating tailwind.config.ts or any other file.' Verify in the agent's handoff note that all three reads were performed.

For frontend-dev and admin-dev

When implementing pages, NEVER use raw <img src='/brand/...'> for the logo. Always import the Logo component from components/brand/logo.tsx. If a variant or size is needed that isn't in the current prop types, extend the component — do not inline a new <Image> tag.

15. Social Media Integration

15.1 Confirmed Handles

BACDA's social presence (verified during research):

Platform	Handle / URL	Notes
Facebook	facebook.com/BayAreaCreativeDanceAcademy	Primary social channel — most active
YouTube	youtube.com/channel/UCPYZ8dOpCwy-bFLRqoiX90g	Channel ID confirmed. Use for video embeds + feed
Instagram	TO BE CONFIRMED by client	Client must provide handle — do not guess from search results
TikTok (optional)	TO BE CONFIRMED by client	If exists
Email	contactus@bayareacreativedancers.org	Primary contact channel

Open question flagged to client

Before Wave 2 starts, BACDA must confirm the Instagram handle (if any) and whether additional platforms exist. Do not deploy with hardcoded guesses. If no Instagram exists, the agent removes

Platform	Handle / URL	Notes
the Instagram icon from the nav/footer rather than linking to a wrong account.		

15.2 Integration Objectives

Social integration is not just icons in the footer. This site should feel alive — reflecting that BACDA is actively producing work. The strategy has three tiers of integration depth:

Tier	Mechanism	Freshness	Complexity
Tier 1 — Static links	Icon links in nav and footer	Never stale	Trivial
Tier 2 — Embedded feeds	YouTube channel feed, Instagram grid, Facebook events	Updates on page load / ISR revalidate	Medium
Tier 3 — Server-fetched + cached	Fetch at build time + Edge revalidation	60s revalidate	Medium-high

This project uses all three tiers depending on the platform's API availability.

15.3 YouTube Integration (Tier 3 — Deep)

15.3.1 Strategy

YouTube is the most important integration because BACDA's core output is performance video. Use the official YouTube Data API v3 to fetch the channel's latest videos at build time and on revalidation. Do NOT embed the channel via iframe only — that breaks performance and SEO.

15.3.2 Implementation

```
// lib/integrations/youtube.ts
import { cache } from 'react';

const CHANNEL_ID = process.env.YOUTUBE_CHANNEL_ID!;  // UCPYZ8dOpCwy-bFLRqoiX90g
const API_KEY    = process.env.YOUTUBE_API_KEY!;

export type YouTubeVideo = {
```

```
  id: string;
  title: string;
  description: string;
  publishedAt: string;
  thumbnails: { high: string; maxres?: string };
  duration?: string;
  viewCount?: string;
};

export const getLatestVideos = cache(async (max = 12): Promise<YouTubeVideo[]> => {
  // Step 1: fetch channel uploads playlist ID (cached forever — it's stable)
  const chanUrl =
`https://www.googleapis.com/youtube/v3/channels?part=contentDetails&id=${CHANNEL_ID}&key=$
{API_KEY}`;
  const chan = await fetch(chanUrl, { next: { revalidate: 86400 } }).then(r => r.json());
  const uploadsPlaylistId = chan.items[0].contentDetails.relatedPlaylists.uploads;

  // Step 2: fetch latest N videos from that playlist
  const listUrl =
`https://www.googleapis.com/youtube/v3/playlistItems?part=snippet&maxResults=${max}&playli
stId=${uploadsPlaylistId}&key=${API_KEY}`;
  const list = await fetch(listUrl, { next: { revalidate: 3600 } }).then(r => r.json());

  // Step 3: enrich with statistics and duration
  const ids = list.items.map((i: any) => i.snippet.resourceId.videoId).join(',');
  const detailUrl =
`https://www.googleapis.com/youtube/v3/videos?part=contentDetails,statistics&id=${ids}&key
=${API_KEY}`;
  const details = await fetch(detailUrl, { next: { revalidate: 3600 } }).then(r =>
r.json());

  const statsById = new Map(details.items.map((d: any) => [d.id, d]));

  return list.items.map((item: any) => {
    const s = item.snippet;
    const stat = statsById.get(s.resourceId.videoId) as any;
    return {
      id: s.resourceId.videoId,
      title: s.title,
      description: s.description,
      publishedAt: s.publishedAt,
      thumbnails: {
        high: s.thumbnails.high?.url,
        maxres: s.thumbnails.maxres?.url,
      },
      duration: stat?.contentDetails?.duration,
      viewCount: stat?.statistics?.viewCount,
    };
  });
});
```

15.3.3 Quota Management

- YouTube Data API free quota: 10,000 units/day

- Channel lookup: 1 unit. PlaylistItems: 1 unit per call. Videos details: 1 unit per call. Total per revalidation: ~3 units
- With 1-hour revalidation on /gallery, that's 72 units/day — well within quota
- Server-side only: NEVER expose YOUTUBE_API_KEY to the client

15.3.4 YouTube UI Components

- components/social/youtube-grid.tsx — grid of video cards with thumbnail, title, duration overlay, published date
- components/social/youtube-player.tsx — lite embed using lite-youtube-embed (performance-critical, reduces iframe weight by 224x)
- components/social/latest-videos-strip.tsx — horizontal scroll strip of 6 most recent videos for home page

lite-youtube-embed — mandatory

Full YouTube iframe embeds add 500KB+ and block main thread. Use the lite-youtube-embed library: it shows the thumbnail as a static image and only loads the full iframe on click. This is non-negotiable for Lighthouse ≥ 90.

15.4 Instagram Integration (Tier 2)

15.4.1 Strategy

Instagram Graph API requires a Facebook Business account + app review for the /media endpoint. For a small non-profit this is often more friction than it's worth. Use a pragmatic alternative:

89. Primary approach: Instagram Basic Display API — simpler OAuth flow, fetches the account's own media. Requires one-time token generation and a 60-day refresh cycle.
90. Fallback: Manual curation — admin pastes Instagram post URLs into a Supabase table 'instagram_highlights'. Build-time fetches oEmbed metadata. No API token needed.
91. Anti-pattern: Embedding the full <iframe> Instagram widget. Loads 400KB+ of Meta tracking code. Reject.

Given BACDA's volunteer-run nature, start with the manual curation approach (fallback). The admin dashboard gets an 'Instagram Highlights' page where they paste post URLs. Graduate to Basic Display API only if the manual approach becomes tedious.

15.4.2 Implementation (Manual Curation)

```
-- Migration: instagram_highlights table
create table instagram_highlights (
  id          uuid primary key default gen_random_uuid(),
  post_url    text not null unique,
  caption     text,
  thumbnail_url text,       -- fetched via oEmbed at save time
  author      text,
  posted_at   timestamptz,
  sort_order  int default 0,
```

```
  is_featured boolean default false,
  created_at  timestamptz default now()
);

alter table instagram_highlights enable row level security;
create policy "public reads ig" on instagram_highlights for select using (true);
create policy "admin writes ig" on instagram_highlights
  for all using (auth.role() = 'authenticated')
  with check (auth.role() = 'authenticated');
```

```
// lib/integrations/instagram.ts
// Called server-side when admin submits a new post URL
export async function fetchOEmbed(postUrl: string) {
  // Instagram oEmbed endpoint (public, no auth for post URLs)
  const url =
`https://graph.facebook.com/v18.0/instagram_oembed?url=${encodeURIComponent(postUrl)}&acce
ss_token=${process.env.FB_OEMBED_TOKEN}`;
  const res = await fetch(url);
  if (!res.ok) throw new Error('oEmbed failed');
  return res.json();
  // returns: { author_name, thumbnail_url, title, ... }
}
```

15.4.3 Instagram UI Components

- components/social/instagram-grid.tsx — 6-tile square grid with hover overlay showing caption
- Clicking a tile opens the post in Instagram (new tab, rel=noopener)
- Placed on /gallery page and optionally as a home-page strip
- Label above grid: 'Follow @bacda on Instagram' → link to profile

15.5 Facebook Integration (Tier 2)

15.5.1 Strategy

Facebook's Page Plugin still works and is acceptable for a page feed. But for performance, prefer Facebook Events API to show upcoming events inline rather than embedding the entire feed.

15.5.2 Implementation

- Footer: static 'Follow on Facebook' link to facebook.com/BayAreaCreativeDanceAcademy
- Events page: if client grants Page access token, fetch upcoming Facebook events via Graph API and merge with Supabase-managed events
- Avoid Facebook's full Page Plugin iframe — loads 600KB+ of Meta SDK. Reject unless explicitly requested
- Share buttons on event detail pages: native Web Share API on mobile, custom anchor tags linking to sharer.php on desktop (no Facebook SDK)

15.5.3 Facebook Share Links (no SDK)

```
// components/social/share-buttons.tsx
function shareUrl(platform: 'facebook' | 'twitter' | 'whatsapp', url: string, text:
string) {
  const enc = encodeURIComponent;
  switch (platform) {
    case 'facebook':
      return `https://www.facebook.com/sharer/sharer.php?u=${enc(url)}`;
    case 'twitter':
      return `https://twitter.com/intent/tweet?url=${enc(url)}&text=${enc(text)}`;
    case 'whatsapp':
      return `https://wa.me/?text=${enc(text + ' ' + url)}`;
  }
}

// Native mobile share
if (navigator.share) {
  await navigator.share({ title, text, url });
}
```

15.6 Social Touchpoint Map Across Pages

Page	Social Touchpoints
Nav (all pages)	Icon row: FB, IG, YouTube (confirmed handles only)
Home	Latest videos strip (YouTube), Instagram grid (6 tiles), Facebook events merge, newsletter signup
About	Follow CTA in team bio, 'See Dalia's journey' → YouTube channel link
Events	Share buttons on each card, 'RSVP via Facebook' for events with FB event link
Event Detail	Full share button row (FB/Twitter/WhatsApp/Email/Copy), embedded YouTube highlight reel if past event
Gallery	YouTube grid + Instagram highlights grid + lightbox photo grid
Testimonials	'Leave a review' CTA → Google Business Profile (if exists) + Facebook page reviews
Contact	All social icons repeated, 'DM us on Instagram' prompt, embedded map
Footer (all pages)	Icon row, newsletter inline form, copyright

15.7 Admin Management

- /admin/social — new admin screen to manage social handles (single row in 'social_config' table)
- /admin/social — Instagram highlights CRUD (paste URL, auto-fetch oEmbed, reorder)

- /admin/settings → 'API Keys' section — rotate YouTube API key, show quota remaining
- Environment variables for all social tokens (see Section 8.4, append: FB_OEMBED_TOKEN, INSTAGRAM_ACCESS_TOKEN if upgraded)

16. SEO & Performance Foundations

16.1 Philosophy

SEO is not a post-launch activity for this project. It is engineered into the architecture from the first commit. The old site ranks poorly because of static HTML with no metadata, no schema, no sitemap, and no structured data. The new site corrects every one of these at the framework level.

Target outcome

Within 90 days of launch: rank in top 3 Google results for 'Bay Area Creative Dancers', 'BACDA dance Bay Area', 'Dalia Sen choreographer'. Within 180 days: rank top 10 for 'Indian classical dance Bay Area', 'Bengali cultural dance San Francisco', and long-tail event queries.

16.2 Technical SEO Checklist

Item	Implementation
robots.txt	app/robots.ts — allow all, sitemap reference
sitemap.xml	app/sitemap.ts — dynamic, pulls from Supabase events + static routes
Canonical URLs	Next.js metadata.alternates.canonical on every page
OpenGraph tags	per-page og:title, og:description, og:image, og:type
Twitter Card	summary_large_image with proper image dims
JSON-LD structured data	Organization, Event, VideoObject, Person, BreadcrumbList
Semantic HTML	header, nav, main, article, section, aside, footer
Heading hierarchy	exactly one h1 per page, sequential h2/h3
Image alt text	enforced via admin CMS — required field
Internal linking	related events, breadcrumbs, footer sitemap
404 page	custom app/not-found.tsx with search + recent events
301 redirects	legacy .html URLs → new routes (see Section 8.3)
HTTPS only	Netlify forces HTTPS, HSTS header set
Mobile-friendly	responsive breakpoints, tap targets ≥ 44px
Page speed (LCP)	≤ 2.5s on 4G mobile
CLS	≤ 0.1

Item	Implementation
INP	≤ 200ms

16.3 Structured Data (JSON-LD)

Every page emits JSON-LD for the most relevant schema.org types. Example schemas required:

16.3.1 Organization (site-wide, in layout.tsx)

```
{
  "@context": "https://schema.org",
  "@type": "PerformingGroup",
  "name": "Bay Area Creative Dancers",
  "alternateName": "BACDA",
  "url": "https://bayareacreativedancers.org",
  "logo": "https://bayareacreativedancers.org/brand/bacda-logo.svg",
  "description": "A San Francisco Bay Area non-profit dance organization led by artistic
director Dalia Sen. BACDA produces classical, contemporary, and fusion Indian dance
performances.",
  "foundingDate": "2018",
  "founder": {
    "@type": "Person",
    "name": "Dalia Sen",
    "jobTitle": "Artistic Director"
  },
  "location": {
    "@type": "Place",
    "address": {
      "@type": "PostalAddress",
      "addressRegion": "CA",
      "addressCountry": "US",
      "addressLocality": "San Francisco Bay Area"
    }
  },
  "email": "contactus@bayareacreativedancers.org",
  "sameAs": [
    "https://www.facebook.com/BayAreaCreativeDanceAcademy",
    "https://www.youtube.com/channel/UCPYZ8dOpCwy-bFLRqoiX90g"
  ]
}
```

16.3.2 Event (on /events and /events/[slug])

```
{
  "@context": "https://schema.org",
  "@type": "DanceEvent",
  "name": "Kingdom Of Dreams",
  "startDate": "2025-09-15T19:00:00-07:00",
  "endDate": "2025-09-15T22:00:00-07:00",
  "eventStatus": "https://schema.org/EventScheduled",
```

```
  "eventAttendanceMode": "https://schema.org/OfflineEventAttendanceMode",
  "location": {
    "@type": "Place",
    "name": "Venue Name",
    "address": "Venue Address"
  },
  "image": ["https://.../og-image.png"],
  "description": "...",
  "organizer": { "@type": "Organization", "name": "Bay Area Creative Dancers" },
  "offers": {
    "@type": "Offer",
    "url": "https://tugoz.com/bacda2025",
    "availability": "https://schema.org/InStock",
    "validFrom": "2025-01-01T00:00:00-08:00"
  },
  "performer": [{ "@type": "PerformingGroup", "name": "Bay Area Creative Dancers" }]
}
```

16.3.3 Person (Dalia Sen and team — on /about)

```
{
  "@context": "https://schema.org",
  "@type": "Person",
  "name": "Dalia Sen",
  "jobTitle": "Artistic Director, Choreographer",
  "affiliation": { "@type": "Organization", "name": "Bay Area Creative Dancers" },
  "description": "...",
  "knowsAbout": ["Classical Indian Dance", "Contemporary Dance", "Choreography",
"Bharatnatyam", "Kathak"],
  "image": "https://.../dalia-sen.jpg"
}
```

16.3.4 VideoObject (for YouTube embeds)

```
{
  "@context": "https://schema.org",
  "@type": "VideoObject",
  "name": "Video title",
  "description": "...",
  "thumbnailUrl": "https://i.ytimg.com/...",
  "uploadDate": "2024-03-15",
  "embedUrl": "https://www.youtube.com/embed/VIDEO_ID",
  "duration": "PT3M45S"
}
```

16.3.5 BreadcrumbList (on every non-home page)

```
{
  "@context": "https://schema.org",
  "@type": "BreadcrumbList",
  "itemListElement": [
```

```
    { "@type": "ListItem", "position": 1, "name": "Home", "item":
"https://bayareacreativedancers.org" },
    { "@type": "ListItem", "position": 2, "name": "Events", "item":
"https://bayareacreativedancers.org/events" },
    { "@type": "ListItem", "position": 3, "name": "Kingdom Of Dreams" }
  ]
}
```

16.4 Metadata Strategy

Every page exports a typed Metadata object. Dynamic pages use generateMetadata() to pull data from Supabase.

```
// app/(public)/events/[slug]/page.tsx
import type { Metadata } from 'next';
import { getEventBySlug } from '@/lib/fetchers/events';

export async function generateMetadata({ params }: { params: { slug: string } }):
Promise<Metadata> {
  const event = await getEventBySlug(params.slug);
  if (!event) return { title: 'Event not found' };

  return {
    title: event.title,
    description: event.subtitle ?? event.description?.slice(0, 160),
    alternates: { canonical: `/events/${event.slug}` },
    openGraph: {
      title: event.title,
      description: event.subtitle ?? '',
      images: event.poster_url ? [event.poster_url] : ['/og-image.png'],
      type: 'article',
      publishedTime: event.created_at,
    },
    twitter: {
      card: 'summary_large_image',
      title: event.title,
      images: event.poster_url ? [event.poster_url] : ['/og-image.png'],
    },
  };
}
```

16.5 Dynamic Sitemap

```
// app/sitemap.ts
import type { MetadataRoute } from 'next';
import { getAllEvents } from '@/lib/fetchers/events';

export default async function sitemap(): Promise<MetadataRoute.Sitemap> {
  const events = await getAllEvents();
  const base = 'https://bayareacreativedancers.org';
```

```
  const staticRoutes = [
    { url: base,                 lastModified: new Date(), changeFrequency: 'weekly' as
const,  priority: 1.0 },
    { url: `${base}/about`,       lastModified: new Date(), changeFrequency: 'monthly' as
const, priority: 0.8 },
    { url: `${base}/events`,      lastModified: new Date(), changeFrequency: 'weekly' as
const,  priority: 0.9 },
    { url: `${base}/gallery`,     lastModified: new Date(), changeFrequency: 'weekly' as
const,  priority: 0.7 },
    { url: `${base}/testimonials`,lastModified: new Date(), changeFrequency: 'monthly' as
const, priority: 0.5 },
    { url: `${base}/contact`,     lastModified: new Date(), changeFrequency: 'yearly' as
const,  priority: 0.6 },
  ];

  const eventRoutes = events.map(e => ({
    url: `${base}/events/${e.slug}`,
    lastModified: new Date(e.updated_at),
    changeFrequency: 'monthly' as const,
    priority: 0.8,
  }));

  return [...staticRoutes, ...eventRoutes];
}

// app/robots.ts
import type { MetadataRoute } from 'next';
export default function robots(): MetadataRoute.Robots {
  return {
    rules: { userAgent: '*', allow: '/', disallow: ['/admin', '/api'] },
    sitemap: 'https://bayareacreativedancers.org/sitemap.xml',
    host: 'https://bayareacreativedancers.org',
  };
}
```

16.6 Core Web Vitals Targets

Metric	Target (p75)	Strategy
LCP (Largest Contentful Paint)	≤ 2.5s	Preload hero image, next/image with priority, CDN
CLS (Cumulative Layout Shift)	≤ 0.1	Explicit width/height on all images, reserve space for fonts
INP (Interaction to Next Paint)	≤ 200ms	RSC where possible, minimize client JS, defer non-critical
TTFB (Time to First Byte)	≤ 600ms	Netlify Edge, ISR cache, avoid serverless cold starts
FCP (First Contentful Paint)	≤ 1.5s	Inline critical CSS, preconnect to fonts domain

Metric	Target (p75)	Strategy
TBT (Total Blocking Time)	≤ 200ms	Code-split, lazy-load below-fold components

16.7 Performance Tactics

- Images: next/image with automatic AVIF/WebP conversion, responsive srcset, lazy loading below the fold
- Fonts: next/font/google with display: swap, subset to 'latin', preload display font only
- JavaScript: React Server Components by default, 'use client' only when needed, dynamic import for heavy components (carousels, lightboxes)
- CSS: Tailwind JIT purges unused classes, critical CSS inlined automatically by Next.js
- Third-party scripts: lite-youtube-embed for YouTube, no Google Analytics by default (use Plausible or Vercel Analytics — cookieless)
- Caching: Netlify CDN handles static assets; ISR with revalidate: 60 for event pages; stale-while-revalidate for API routes
- Preconnect: <link rel='preconnect'> to fonts.gstatic.com, i.ytimg.com, supabase storage CDN
- HTTP/2 push + Brotli compression: enabled by default on Netlify

16.8 Content SEO

16.8.1 Keyword Strategy

Target Keyword	Page	Search Intent
Bay Area Creative Dancers	/	Navigational — brand
BACDA dance	/	Navigational — brand
Dalia Sen choreographer	/about	Informational — person
Indian classical dance Bay Area	/	Informational — service
Bengali dance San Francisco	/events	Informational — service
NABC dance performance	/events?year=2017	Informational — historical
Bharatnatyam workshop Bay Area	/events	Transactional — workshop
Indian dance classes Fremont / Pleasanton	/contact	Transactional — local
Kingdom of Dreams BACDA 2025	/events/kingdom-of-dreams-2025	Transactional — tickets

16.8.2 Copy Guidelines

- Every page has a unique H1 with the primary keyword naturally placed
- First 100 words of each page contain the primary keyword without stuffing
- Meta descriptions 140-160 chars, compelling, action-oriented — not keyword dumps
- Event descriptions written in markdown with proper headings (h2/h3) and paragraph structure
- Image captions provide context — these appear in Google Image search
- Internal linking: every event links to related past performances, every bio links to events they were part of

16.9 Local SEO

- Schema PostalAddress with addressRegion: CA, addressLocality: Bay Area (or specific city)
- Google Business Profile — create/claim, link from site, embed reviews on /testimonials
- Bing Places — claim parallel listing
- NAP consistency (Name, Address, Phone) — identical across site, GBP, Facebook, YouTube About
- Local event schema includes venue with full address, improves Google Events appearance

16.10 Analytics & Search Console

- Google Search Console — verify via DNS TXT record (not HTML file — avoid clutter)
- Submit sitemap.xml immediately after launch
- Monitor: Core Web Vitals report, Coverage (indexing), Performance (queries/clicks)
- Bing Webmaster Tools — same sitemap submission
- Analytics: Plausible (privacy-first, cookieless, $9/mo) or Vercel Analytics (free)
- Do NOT use Google Analytics 4 — requires cookie banner, hurts CWV, over-complex for this use case
- Event tracking: outbound clicks (ticket links, social icons), contact form submit, newsletter signup

16.11 Accessibility = SEO

Google increasingly uses accessibility signals as ranking inputs, and accessibility requirements overlap ~80% with technical SEO. Compliance is already specified in Section 6.7 — this section reiterates the overlap:

- Semantic HTML → Google understands content structure
- Alt text on images → image search + screen readers
- Descriptive link text (never 'click here') → crawl context + screen readers
- Proper heading hierarchy → content outline for both Google and AT
- Color contrast ≥ 4.5:1 → Google Lighthouse score + readability

16.12 SEO Agent Responsibilities

Add a new specialist to the agent roster (update Section 9.2):

Agent	Role	Primary outputs
seo-agent	Metadata, schema, sitemap, robots, keyword audits, CWV monitoring	app/sitemap.ts, app/robots.ts, JSON-LD helpers in lib/seo/, per-page metadata exports

The seo-agent runs in Wave 2 alongside frontend-dev, reviewing every page PR for: metadata completeness, canonical tag presence, heading hierarchy, alt text coverage, and schema validity. SEO is not a separate phase — it is a review gate on every merge.

16.13 Pre-Launch SEO Checklist

92. Every page has unique <title> and <meta description> under 60/160 chars
93. Every page has canonical URL set
94. OpenGraph image exists and is 1200×630
95. sitemap.xml generates and includes all public routes
96. robots.txt allows all public pages, blocks /admin and /api
97. All JSON-LD validates at schema.org validator and Google Rich Results Test
98. All images have alt text (enforced at admin level — no save without alt)
99. Lighthouse SEO score = 100
100. Lighthouse Performance score (mobile) ≥ 90
101. Lighthouse Accessibility score ≥ 95
102. 301 redirects in place from all legacy .html URLs
103. Google Search Console verified, sitemap submitted
104. Bing Webmaster Tools verified, sitemap submitted
105. Plausible or Vercel Analytics installed and tracking
106. Google Business Profile created, linked from site
107. Social handles verified and linked (Section 15.1 confirmed by client)

Appendix A — Starter File Structure

```
bacda-site/
├── .github/
│   └── workflows/
│       └── ci.yml
├── .agent-handoffs/          # agent handoff notes (gitignored in final prod)
├── app/
│   ├── (public)/
│   │   ├── layout.tsx
│   │   ├── page.tsx
│   │   ├── about/page.tsx
│   │   ├── events/
│   │   │   ├── page.tsx
│   │   │   └── [slug]/page.tsx
│   │   ├── gallery/page.tsx
│   │   ├── testimonials/page.tsx
│   │   └── contact/page.tsx
│   ├── (admin)/
│   │   ├── layout.tsx
│   │   ├── admin/
│   │   │   ├── page.tsx              # dashboard
│   │   │   ├── login/page.tsx
│   │   │   ├── home/page.tsx
│   │   │   ├── events/
│   │   │   │   ├── page.tsx
│   │   │   │   └── [id]/page.tsx
│   │   │   ├── testimonials/page.tsx
│   │   │   ├── team/page.tsx
│   │   │   ├── media/page.tsx
│   │   │   ├── submissions/page.tsx
│   │   │   ├── newsletter/page.tsx
│   │   │   └── settings/page.tsx
│   ├── api/
│   │   └── contact/route.ts
│   ├── globals.css
│   └── middleware.ts             # supabase session + admin protection
├── components/
│   ├── ui/                       # shadcn primitives
│   └── sections/                 # page sections
├── lib/
│   ├── supabase/
│   │   ├── client.ts
│   │   ├── server.ts
│   │   └── middleware.ts
│   ├── fetchers/                 # typed data fetchers
│   ├── validators/               # Zod schemas
│   └── integrations/             # youtube, resend, etc.
├── supabase/
│   ├── migrations/
│   │   ├── 0001_events.sql
│   │   ├── 0002_event_photos.sql
│   │   ├── 0003_event_videos.sql
│   │   ├── 0004_testimonials.sql
```

```
│   │   ├── 0005_team_members.sql
│   │   ├── 0006_home_content.sql
│   │   ├── 0007_gallery_videos.sql
│   │   ├── 0008_contact_submissions.sql
│   │   └── 0009_newsletter_subscribers.sql
│   ├── seed.sql
│   └── functions/
│       └── send-contact-email/
│           └── index.ts
├── __tests__/
├── e2e/
├── public/
├── CLAUDE.md                      # root agent constitution
├── TASKS.md                       # orchestrator-generated ticket list
├── README.md
├── README_ADMIN.md
├── ARCHITECTURE.md
├── RUNBOOK.md
├── netlify.toml
├── next.config.js
├── tailwind.config.ts
├── tsconfig.json
├── package.json
└── .env.example
```

Appendix B — Seed Data Inventory

Content to be preloaded into Supabase during Wave 1 via seed.sql.

Events — Past (archive)

- Tasher Desh (First Production)
- NABC 2009 Opening Ceremony
- NABC 2012 Closing Ceremony — Las Vegas
- OMG — Oh My God! (2014)
- Kalamahotsav 2014 Opening Ceremony
- Chirosokha — Durga Pujo 2015
- PM Modi Bay Area Inaugural Show (Sept 2015, SAP Center San Jose)
- Chitra — EnActe 2017
- NABC 2017 Opening Ceremony
- NABC 2017 Closing Ceremony
- Dance Workshop — Sanjib Bhattacharya (Navanritya)
- Sitar Concert — Pt. Sugato Nag
- Raabdta — Maiden BACDA Production
- Spring India Festival — Union Square, San Francisco
- Folk Dance — Agomoni Nrityamela (Aug 24, 2019, Dougherty Valley High School)
- West Coast Theater Festival (Aug 31 – Sep 1, 2019, Amador Valley High School)
- Banga Mela 2019 (Sep 7-8, 2019, Tempe Center for the Arts)
- Bodhayon – the Awakening (Durga Puja 2020)
- Tanushree Shankar Workshop (Feb 2023)
- Mallika Sarabhai Workshop (April 2023)
- BACDA Show with Kumar Sharma (2023)

Events — Upcoming

- Kingdom Of Dreams (2025) — ticketing at tugoz.com/bacda2025

Team Members

- Dalia Sen — Artistic Director (lead) — credits: NABC 2009, NABC 2012, NABC 2017, Chitra/EnActe, Kalamahotsav 2014/2015, PM Modi Opening 2015, Raabdta, Bodhayon
- Urmi Dutta — Coordinator
- Dalia Chatterjee Sen — Coordinator
- Ishita Sinha — Coordinator

Frequent Collaborators (mention in event records)

- Tanushree Shankar, Mallika Sarabhai, Debojyoti Mishra, Sudarshan Chakroborty, Sanjib Bhattacharya
- Milly Roy, Surajit Sengupta, Subhen Khatua, Ranjita Chakraborty, Ballori Sengupta, Anuradha Mookerjee, Poushali Banerjee, Sruti Das, Sudeshna Chatterjee

Appendix C — Glossary

- **ISR** — Incremental Static Regeneration. Next.js feature that re-renders pages in the background on a schedule, keeping content fresh without a full rebuild.
- **RLS** — Row Level Security. Postgres feature used by Supabase to enforce per-row access rules based on the authenticated user.
- **App Router** — Next.js 14 file-system-based routing using the app/ directory with React Server Components.
- **shadcn/ui** — Copy-paste component library built on Radix primitives and Tailwind. Not a dependency — components live in your repo.
- **Wave** — A batch of agent tasks that can run in parallel. Waves execute sequentially; tasks within a wave execute concurrently.
- **CLAUDE.md** — Per-directory context file read by Claude-powered agents. Declares scope, patterns, and constraints for that agent.